Universidad Politécnica de Chiapas

Ingeniería en Tecnologías de la Información e Innovación Digital

Bases de Datos en la Nube

Medición de métricas en bases documentales

MongoDB frente a PostgreSQL JSONB bajo contención de CPU y memoria (Docker cgroups v2)

Alumno: Luis Ángel Pérez Aguilera

Grupo: 7B

Docente: Horacio Irán Solís Cisneros

1 de noviembre de 2026

Resumen

Se comparan MongoDB 9.0.4 y PostgreSQL 18.6 (JSONB) al ejecutar dos agregaciones sobre 500 000 documentos JSON con tres niveles de anidamiento, bajo tres perfiles de recursos impuestos con cgroups v2: A (4 CPU / 4 GB), B (2 CPU / 2 GB) y C (1 CPU / 1 GB). Cada celda de la matriz (2 motores × 3 perfiles × 2 consultas × 3 concurrencias, más las dos variantes de memoria de Q2) se midió tres veces con el ciclo *flush* → warmup 60 s → medición 300 s → cooldown 60 s: 162 corridas en total. La agregación con desanidado (Q1) separa a los motores: MongoDB la resuelve en flujo con ∼128 KB de memoria de agrupación y queda limitada solo por CPU, mientras que PostgreSQL ordena filas completas con *external merge*, escribe del orden de 1 GB temporal por consulta y, con 1 GB y 50 clientes, termina en OOM. El Top-1000 (Q2) no produce spill en ningún motor ni variante: ambos lo resuelven con un ordenamiento top-k de menos de 1,1 MB, y el límite de CPU domina su latencia.

Índice

1. Especificaciones del host y del entorno Docker **2**
- 1.1. Equipos . . . 2
- 1.2. Motores, imágenes y configuración . . . 2
- 1.3. Perfiles de recursos y límites reales verificados . . . 2

2. Método de medición y verificación del baseline **3**
- 2.1. Dataset . . . 3
- 2.2. Consultas . . . 3
- 2.3. Protocolo . . . 4
- 2.4. Métricas y estado de cada corrida . . . 4
- 2.5. Resumen de repeticiones . . . 5
- 2.6. Verificación del baseline . . . 5

3. Resultados consolidados **5**

4. Análisis gráfico **8**

5. Planes de ejecución **10**

6. Interpretación: memoria, volcado a disco y throttling **11**

7. Recomendaciones técnicas para despliegues documentales en la nube **13**

Amenazas a la validez **14**

A. Reproducción y artefactos **15**

1. Especificaciones del host y del entorno Docker

1.1. Equipos

Para completar la matriz en una sola jornada, las mediciones se repartieron en dos rondas simultáneas en dos equipos. La ronda 1 midió los perfiles A y C *en paralelo* en el equipo principal; la ronda 2 midió el perfil B en un segundo equipo. De este modo el baseline (A) y la contención severa (C) comparten hardware, y solo B se midió en otro equipo (sección 7).

Tabla 1: Equipos de medición (datos de `results/entorno/ronda*/host.txt`).

	Ronda 1 (perfiles A y C)	Ronda 2 (perfil B)
CPU	Intel Core i5-13420H, 8 núcleos / 12 hilos	Dell Vostro 3400, Intel Core i5-1135G7 @ 2.40 GHz, 4 núcleos / 8 hilos
RAM del equipo	16 GB (15.7 GB útiles)	8 GB
Sistema	Windows 11 + WSL2 (Ubuntu 24.04), kernel 6.6.87.2-microsoft-standard-WSL2	Windows 11 + WSL2 (Ubuntu), kernel 6.18.54.1-microsoft-standard-WSL2
Memoria de la VM WSL2	7.6 GB (valor por defecto) + 2 GB de swap	4.8 GB (`.wslconfig`) + 2 GB de swap
Docker	Docker Desktop, Engine 29.7.2, cgroup v2	Docker Desktop, Engine 29.9.2, cgroup v2 (driver `cgroupfs`)

1.2. Motores, imágenes y configuración

- **MongoDB 9.0.4** (`mongo:9.0.4`, digest `sha256:452bdaf2...`), motor WiredTiger, colección `resultados_unidad` sin sharding, índices simples en `resultado_id` y `cerrado_en`.
- **PostgreSQL 18.6** (`postgres:18.6`, digest `sha256:3a426452...`), tabla `resultados_unidad` con las columnas `id UUID PRIMARY KEY` y `data JSONB`, PK en `id` y B-Tree sobre la expresión `data->>'cerrado_en'`.
- Ninguno de los dos índices cubre el total del Top-K (`calificacion.final`), como exige la especificación.
- Configuración **por defecto** en ambos motores. WiredTiger dimensiona su caché a partir del límite de memoria del contenedor. PostgreSQL usa `shared_buffers` y `work_mem` (4 MB) por defecto y hasta 2 workers paralelos por consulta. Lo único que se cambia es `work_mem` por sesión en las variantes de Q2.
- Telemetría: cAdvisor v0.53.0, Node Exporter v1.9.1 y Prometheus v3.5.0 con `scrape_interval: 1s`, fuera de los límites de los contenedores medidos.

1.3. Perfiles de recursos y límites reales verificados

Los perfiles se declaran en `infra/docker-compose.yml` con `cpus`, `mem_limit`, `mem_swappiness: 0` y `memswap_limit`. En cgroup v2 la *swappiness* por contenedor no existe y Docker la descarta, así que además se iguala `memswap_limit` a `mem_limit`, lo que deja el swap del contenedor en cero. Antes de cada bloque de corridas, el programa de medición registra los límites efectivos leídos de la propia cgroup (`results/entorno/ronda*/<perfil>_<motor>.txt`):

Tabla 2: Límites declarados y valores leídos de la cgroup v2 de cada contenedor.

Perfil	cpus	mem_limit	cpu.max	memory.max	memory.swap.max
A PROFILE_A_BASELINE	4.0	4096 MB	400000 100000	4 294 967 296	0
B PROFILE_B_MEDIUM	2.0	2048 MB	200000 100000	2 147 483 648	0
C PROFILE_C_SEVERE	1.0	1024 MB	100000 100000	1 073 741 824	0

Cada perfil tiene su propio par de contenedores (MongoDB y PostgreSQL) con sus propios volúmenes de datos. Durante una corrida solo está encendido el motor que se mide, para que la caché del motor inactivo no ocupe memoria del equipo.

2. Método de medición y verificación del baseline

2.1. Dataset

El esquema de ejemplo de la especificación de la actividad (órdenes de compra) se sustituyó por la colección `resultados_unidad` de un proyecto de control académico, conservando lo que exige la especificación: 500 000 documentos JSON, tres niveles de anidamiento (`contexto.materia.nombre`, `calificacion.desglose[].criterio`), un arreglo de 2 a 4 elementos (promedio 3.0), un identificador UUID y una fecha ISO-8601 en formato UTC fijo. El generador (`data/synthetic_data_generator.py`) es determinista (semilla 42); el NDJSON resultante tiene sha256 `10a04043...c429` y se carga idéntico en los dos motores. La tabla 3 muestra qué campo cumple cada papel de las consultas de la especificación.

Tabla 3: Campos de la especificación y campos equivalentes del dataset usado.

Papel en la consulta	Campo en la especificación	Campo usado
Arreglo que se desanida (Q1)	`items[]`	`calificacion.desglose[]`
Filtro numérico (Q1)	`price > 50`	`desglose.promedio > 6`
Subtotal (Q1)	`qty * price`	`porcentaje * promedio`
Claves de agrupación (Q1)	país, categoría	`contexto.materia.nombre`, `desglose.criterio`
Estado (Q2)	`payment.status = completed`	`unidad.estado = 'cerrada'`
Fecha (Q2)	`timestamp >= 2026-01-01`	`cerrado_en >= 2026-01-01T00:00:00Z`
Total del Top-K (Q2)	`payment.final_total`	`calificacion.final`

2.2. Consultas

Q1 desanida el desglose (1 500 160 elementos), filtra los promedios mayores que 6 (974 583 elementos) y agrupa en 180 grupos con suma, promedio y conteo. Q2 filtra las unidades cerradas desde 2026 (212 641 documentos), ordena de forma descendente por la calificación final sin índice y devuelve el Top-1000. El código está en `queries/`; la versión de PostgreSQL:

```
-- Q1
SELECT data->'contexto'->'materia'->>'nombre' AS materia, d->>'criterio' AS criterio,
      SUM((d->>'porcentaje')::numeric * (d->>'promedio')::numeric) AS total_aporte,
      AVG((d->>'porcentaje')::numeric * (d->>'promedio')::numeric) AS avg_aporte,
      COUNT(*) AS count
FROM resultados_unidad, jsonb_array_elements(data->'calificacion'->'desglose') AS d
WHERE (d->>'promedio')::numeric > 6
GROUP BY 1, 2;
-- Q2
SELECT data->>'resultado_id', data->'alumno'->>'id', (data->'calificacion'->>'final')::numeric
FROM resultados_unidad
WHERE data->'unidad'->>'estado' = 'cerrada' AND data->>'cerrado_en' >= '2026-01-01T00:00:00Z'
```

```
ORDER BY (data->'calificacion'->>'final')::numeric DESC
LIMIT 1000;
```

La fecha se compara como texto: el cast a `timestamptz` de la especificación no es inmutable y anula el índice de expresión; con el formato UTC fijo el orden del texto es el cronológico y coincide con la comparación de cadenas de MongoDB. Las variantes de memoria de Q2 se codifican en `query_id` para no agregar columnas al CSV: `Q2_DISK0` / `Q2_DISK1` (`allowDiskUse` false / true) en MongoDB y `Q2_WM4` / `Q2_WM64` (`work_mem` 4 MB / 64 MB) en PostgreSQL.

2.3. Protocolo

- **Matriz.** Una *celda* es una combinación de motor, perfil, consulta y concurrencia; cada celda se mide 3 veces y cada medición es una *corrida*. 2 motores × 3 perfiles × (Q1 + 2 variantes de Q2) × concurrencias 1, 10 y 50 × 3 repeticiones = 162 corridas (54 por perfil), cada una con una fila en el CSV de 14 columnas que fija la especificación.
- **Ciclo por corrida.** `sync; echo 3 > /proc/sys/vm/drop_caches`, warmup de 60 s, medición de 300 s y cooldown de 60 s. El flush se ejecuta en un contenedor privilegiado, que comparte el kernel de la VM. Cada corrida duró 423–424 s según los registros de ejecución (`results/logs/`).
- **Perfiles en paralelo.** Como el flush de caché afecta a todo el kernel, las mediciones de A y C, que corrieron al mismo tiempo en el mismo equipo, se sincronizan antes de cada corrida: cada una espera a la otra, solo una ejecuta el flush y ambas arrancan en el mismo segundo. Así ninguna vacía la caché a la otra a mitad de una medición.
- **Programa de medición** (*runner*, carpeta `runner/`, en Python). Usa un proceso por perfil y simula cada cliente concurrente con un hilo que tiene su propia conexión (pymongo 4.19, psycopg 3.3) y lanza una consulta tras otra sin pausa. Es una desviación menor del "Python multiproceso" de la especificación: los drivers liberan el GIL mientras esperan a la base de datos, y el programa usa ∼150 MB por perfil en lugar de 1–2 GB con 50 procesos.
- **Ventana de medición**. Cada consulta recibe como *timeout* el tiempo que le queda a la ventana (`maxTimeMS` / `statement_timeout`). Latencias y throughput cuentan solo las consultas que empiezan y terminan dentro de los 300 s.

2.4. Métricas y estado de cada corrida

- **Latencia**: percentiles P50, P95 y P99 de las latencias individuales (no promedios); las latencias crudas de cada corrida están en `results/raw/`.
- **Throughput**: consultas terminadas ÷ 300 s.
- **Throttling**: tiempo que el kernel detuvo al contenedor por haber agotado su *cuota* de CPU (el límite `cpus` del perfil). Es el delta de `throttled_usec` de `cpu.stat`, la misma métrica que `container_cpu_cfs_throttled_seconds_total`. Puede superar los 300 s de la ventana porque el kernel suma el tiempo de cada CPU donde había hilos del contenedor esperando.
- **CPU-s**: segundos de CPU consumidos por el contenedor en la ventana. Con la cuota agotada equivalen a CPUs del perfil × 300 s (1 200 en A, 600 en B, 300 en C).
- **RSS máximo**: máximo de `container_memory_rss` (memoria anónima) en la ventana. En PostgreSQL no incluye los buffers compartidos.
- **Spill**: delta de `pg_stat_database.temp_bytes` en PostgreSQL y de los contadores de spill de `serverStatus` en MongoDB.

- **OOM**: delta de `oom_kill` en `memory.events` del contenedor.
- **Estado**: `oom` si hubo OOM; `error` si hubo fallos o ninguna consulta terminó en la ventana; `spill` si hubo volcado a disco; `ok` en otro caso. Las corridas `spill`, `oom` o con fallos guardan evidencia (notas, `memory.events` y log del motor) en `results/evidencia_spill_oom/`.

2.5. Resumen de repeticiones

Cada celda se resume con la **mediana** de sus 3 repeticiones y el mínimo y máximo como rango: con tres valores, una corrida atípica desplaza mucho la media. El consolidado completo está en `report/datos/consolidado.csv` y lo genera `scripts/analyze_results.py`.

2.6. Verificación del baseline

- **Mismo dataset en todos los perfiles**: conteo de 500 000 en ambos motores, índices exigidos y muestra de 25 documentos idéntica en MongoDB, PostgreSQL y el NDJSON (`scripts/validate_data.py`).
- **Mismas consultas**: Q1 y Q2 devuelven el mismo resultado en ambos motores (`scripts/check_queries.py`); en Q1 las sumas se comparan con tolerancia relativa de 10^{-9} porque MongoDB suma en *double* y PostgreSQL en `numeric`, y en Q2 se compara la lista ordenada de totales por los empates.
- **Límites efectivos**: tabla 2, leídos de la cgroup en cada corrida.
- **Baseline sin contención a 1 cliente**: en A, MongoDB registra 0 s de throttling en Q1 y Q2 con un cliente, y la variación entre repeticiones es pequeña (Q1: 0.623–0.637 QPS).
- **Pruebas previas**: antes de la matriz se midió una celda de prueba y después la matriz completa de un perfil con ventanas cortas (warmup 5 s, medición 20 s, cooldown 5 s). Con ellas se validaron el formato del CSV, la sincronización entre perfiles en paralelo y la detección de spill y OOM.

3. Resultados consolidados

Las tablas 4–6 resumen las 18 celdas (54 corridas) de cada perfil. “Estado” cuenta las 3 repeticiones; “–” indica que ninguna consulta terminó dentro de la ventana.

Tabla 4: Resultados consolidados del perfil A: mediana de 3 repeticiones. Latencias en segundos; throttling y RSS máximo de la ventana de medición de 300 s.

Motor	Query	Conc.	Estado	QPS	P50 (s)	P95 (s)	P99 (s)	Throttled (s)	RSS (MB)
MongoDB	Q1	1	3 ok	0.63	1.57	1.62	1.64	0	793
MongoDB	Q1	10	3 ok	0.43	22.95	33.11	34.90	1 937	851
MongoDB	Q1	50	3 ok	0.30	147.76	169.96	173.25	2 351	1 004
MongoDB	Q2_DISK0	1	3 ok	2.27	0.43	0.52	0.58	0	1 011
MongoDB	Q2_DISK0	10	3 ok	3.39	2.77	4.68	5.39	1 864	1 020
MongoDB	Q2_DISK0	50	3 ok	2.07	23.93	31.19	33.43	2 348	1 005
MongoDB	Q2_DISK1	1	3 ok	2.28	0.43	0.47	0.51	0	1 000
MongoDB	Q2_DISK1	10	3 ok	2.84	3.42	4.80	5.35	1 875	1 004
MongoDB	Q2_DISK1	50	3 ok	1.93	24.99	33.55	36.25	2 350	994
PostgreSQL	Q1	1	3 spill	0.16	5.94	9.20	11.26	0	102
PostgreSQL	Q1	10	3 spill	0.06	103.53	167.84	168.66	54	566
PostgreSQL	Q1	50	3 error	0.00	–	–	–	97	1 810
PostgreSQL	Q2_WM4	1	3 ok	7.89	0.12	0.16	0.20	137	14
PostgreSQL	Q2_WM4	10	3 ok	1.45	5.15	12.92	13.86	1 863	43
PostgreSQL	Q2_WM4	50	3 ok	1.51	34.97	43.20	45.55	2 082	126
PostgreSQL	Q2_WM64	1	3 ok	7.85	0.12	0.16	0.20	148	14
PostgreSQL	Q2_WM64	10	3 ok	1.41	5.67	12.97	14.91	1 857	43
PostgreSQL	Q2_WM64	50	3 ok	1.44	36.88	43.49	46.26	2 090	126

Tabla 5: Resultados consolidados del perfil B: mediana de 3 repeticiones. Latencias en segundos; throttling y RSS máximo de la ventana de medición de 300 s.

Motor	Query	Conc.	Estado	QPS	P50 (s)	P95 (s)	P99 (s)	Throttled (s)	RSS (MB)
MongoDB	Q1	1	3 ok	0.39	2.44	4.01	4.14	12	836
MongoDB	Q1	10	3 ok	0.16	58.00	69.17	73.56	1 815	830
MongoDB	Q1	50	3 ok	0.06	292.03	297.40	298.95	1 818	893
MongoDB	Q2_DISK0	1	3 ok	0.60	1.59	2.45	2.64	11	941
MongoDB	Q2_DISK0	10	3 ok	0.53	18.19	21.47	21.71	1 528	867
MongoDB	Q2_DISK0	50	3 ok	0.33	132.41	160.69	168.83	1 779	872
MongoDB	Q2_DISK1	1	3 ok	0.60	1.59	2.15	2.84	12	935
MongoDB	Q2_DISK1	10	3 ok	0.50	18.35	22.85	22.86	1 511	920
MongoDB	Q2_DISK1	50	3 ok	0.32	130.87	163.17	169.22	1 782	872
PostgreSQL	Q1	1	3 spill	0.18	5.63	7.04	7.40	193	109
PostgreSQL	Q1	10	3 spill	0.03	170.54	177.55	177.95	780	572
PostgreSQL	Q1	50	3 error	0.00	–	–	–	1 264	1 796
PostgreSQL	Q2_WM4	1	3 ok	1.38	0.70	0.88	1.72	642	20
PostgreSQL	Q2_WM4	10	3 ok	0.80	10.10	21.10	21.73	1 485	43
PostgreSQL	Q2_WM4	50	3 ok	0.78	60.51	71.27	73.18	1 672	125
PostgreSQL	Q2_WM64	1	3 ok	1.38	0.70	0.88	1.54	622	20
PostgreSQL	Q2_WM64	10	3 ok	0.80	9.99	21.16	21.73	1 487	44
PostgreSQL	Q2_WM64	50	3 ok	0.78	61.06	69.56	71.33	1 681	126

Tabla 6: Resultados consolidados del perfil C: mediana de 3 repeticiones. Latencias en segundos; throttling y RSS máximo de la ventana de medición de 300 s.

Motor	Query	Conc.	Estado	QPS	P50 (s)	P95 (s)	P99 (s)	Throttled (s)	RSS (MB)
MongoDB	Q1	1	3 ok	0.52	1.81	2.23	2.27	62	629
MongoDB	Q1	10	3 ok	0.10	91.15	113.24	113.31	2 893	616
MongoDB	Q1	50	3 error	0.00	–	–	–	3 271	724
MongoDB	Q2_DISK0	1	3 ok	0.22	4.49	5.09	5.39	82	739
MongoDB	Q2_DISK0	10	3 ok	0.20	46.61	62.68	62.77	2 277	683
MongoDB	Q2_DISK0	50	3 ok	0.17	152.87	152.92	152.95	3 082	690
MongoDB	Q2_DISK1	1	3 ok	0.23	4.48	4.62	4.84	78	720
MongoDB	Q2_DISK1	10	3 ok	0.13	57.28	64.33	64.47	2 317	688
MongoDB	Q2_DISK1	50	3 ok	0.17	150.82	150.92	150.98	3 059	700
PostgreSQL	Q1	1	3 spill	0.02	37.09	51.82	51.86	22	101
PostgreSQL	Q1	10	3 error	0.00	–	–	–	103	546
PostgreSQL	Q1	50	3 oom	0.00	–	–	–	360	956
PostgreSQL	Q2_WM4	1	3 ok	1.61	0.60	0.70	0.71	958	16
PostgreSQL	Q2_WM4	10	3 ok	0.40	17.47	42.88	44.07	2 639	43
PostgreSQL	Q2_WM4	50	3 ok	0.40	117.18	154.30	158.60	2 814	124
PostgreSQL	Q2_WM64	1	3 ok	1.59	0.61	0.70	0.78	958	16
PostgreSQL	Q2_WM64	10	3 ok	0.39	16.71	45.26	46.02	2 626	43
PostgreSQL	Q2_WM64	50	3 ok	0.40	125.18	148.37	150.11	2 820	123

Observaciones principales.

- **Q1 en MongoDB** pasa de un P99 de 1.6 s (A) a 2.3 s (C) con un cliente, y de 34.9 s a 113.3 s con 10 clientes. Con 50 clientes, A sostiene 0.30 QPS con un P99 de 173 s, y en C ninguna consulta termina en 300 s.
- **Q1 en PostgreSQL** es la celda más cara: vuelca a disco en todas las corridas en las que termina alguna consulta. Con un cliente, el P99 sube de 11.3 s (A) a 51.9 s (C); con 50 clientes ninguna termina en A ni en B, y en C el contenedor muere por OOM.
- **Q2** termina en todas las corridas de ambos motores. Con un cliente PostgreSQL es el más rápido (7.9 QPS en A frente a 2.3 de MongoDB), y con 10 y 50 clientes MongoDB lo alcanza o supera en A.
- Las variantes de memoria de Q2 dan el mismo resultado dentro del ruido entre repeticiones.
- **Perfil B** (2 CPU / 2 GB, en el otro equipo) repite el patrón de estados de A: MongoDB y Q2 terminan en las 45 corridas, y PostgreSQL Q1 vuelca a disco con 1 y 10 clientes y no termina ninguna consulta con 50, sin OOM aunque su RSS llega a 1.8 GB de 2 GB. MongoDB Q1 con 50 clientes sí termina en B (0.06 QPS, P99 de 299 s, al borde de la ventana), a diferencia de C.
- **B no siempre queda entre A y C.** Con 10 y 50 clientes sí: el P99 de MongoDB Q1 con 10 clientes es 34.9 s en A, 73.6 s en B y 113.3 s en C. Con un cliente, en cambio, B es a veces más lento que C (MongoDB Q1: 4.1 s frente a 2.3 s; PostgreSQL Q2: 1.5–1.7 s frente a 0.7–0.8 s). Con un cliente casi no hay throttling y la latencia la fija la velocidad de cada núcleo, menor en el i5 de 11.ª generación (sección 7).

Tabla 7: Celdas con spill, error u OOM. Spill: mediana de `temp_bytes` escritos en la ventana; `oom_kill`: suma de las 3 repeticiones.

Perfil	Motor / query	Conc.	Estado (3 rep.)	Spill (GB/ventana)	oom_kill
A	PostgreSQL Q1	1	3 spill	42.5	0
A	PostgreSQL Q1	10	3 spill	22.9	0
A	PostgreSQL Q1	50	3 error	33.5	0
B	PostgreSQL Q1	1	3 spill	47.6	0
B	PostgreSQL Q1	10	3 spill	17.0	0
B	PostgreSQL Q1	50	3 error	13.5	0
C	MongoDB Q1	50	3 error	–	0
C	PostgreSQL Q1	1	3 spill	6.7	0
C	PostgreSQL Q1	10	3 error	7.1	0
C	PostgreSQL Q1	50	3 oom	–	28

4. Análisis gráfico

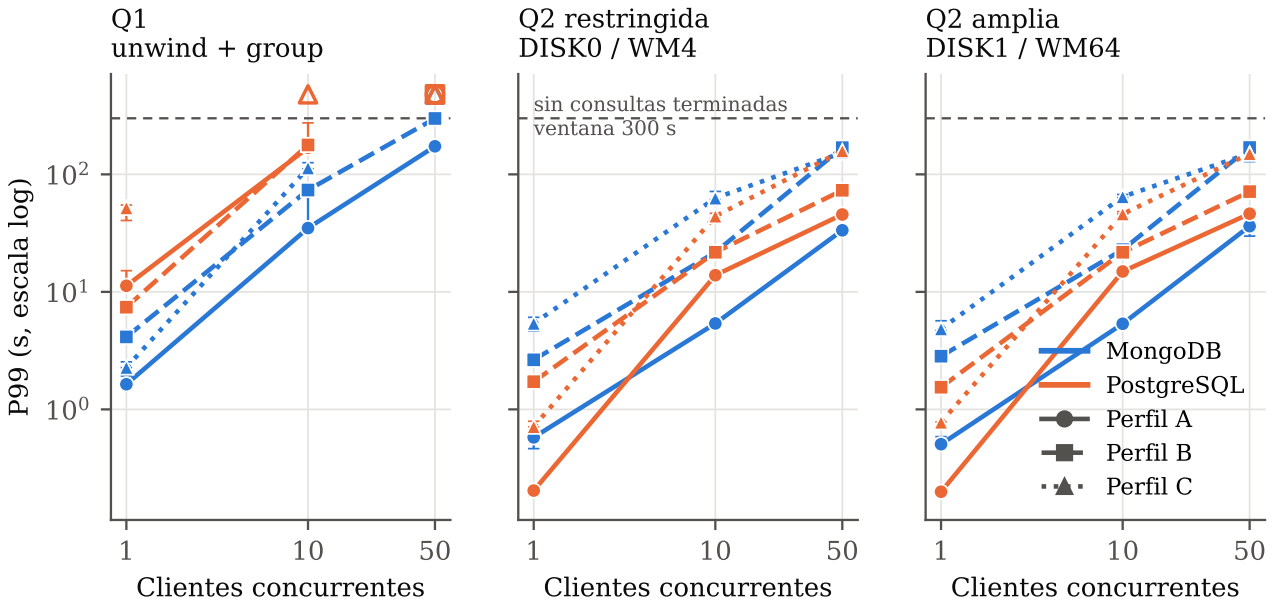


Figura 1: P99 frente a concurrencia (ejes logarítmicos), mediana de 3 repeticiones con el rango mínimo–máximo. Color: motor; marcador y trazo: perfil. Los marcadores huecos sobre la línea de 300 s son celdas en las que ninguna consulta terminó dentro de la ventana (latencia censurada).

La figura 1 muestra que el P99 crece casi en proporción a la concurrencia en cuanto la CPU se satura. Pasar de 1 a 10 clientes multiplica el P99 de Q1 en MongoDB por ∼21 en A y por ∼50 en C. En Q2 la pendiente de PostgreSQL es más fuerte: parte de 0.2 s, pero con 10 clientes ya compite con MongoDB porque cada consulta usa hasta 3 procesos y satura la cuota con un solo cliente.

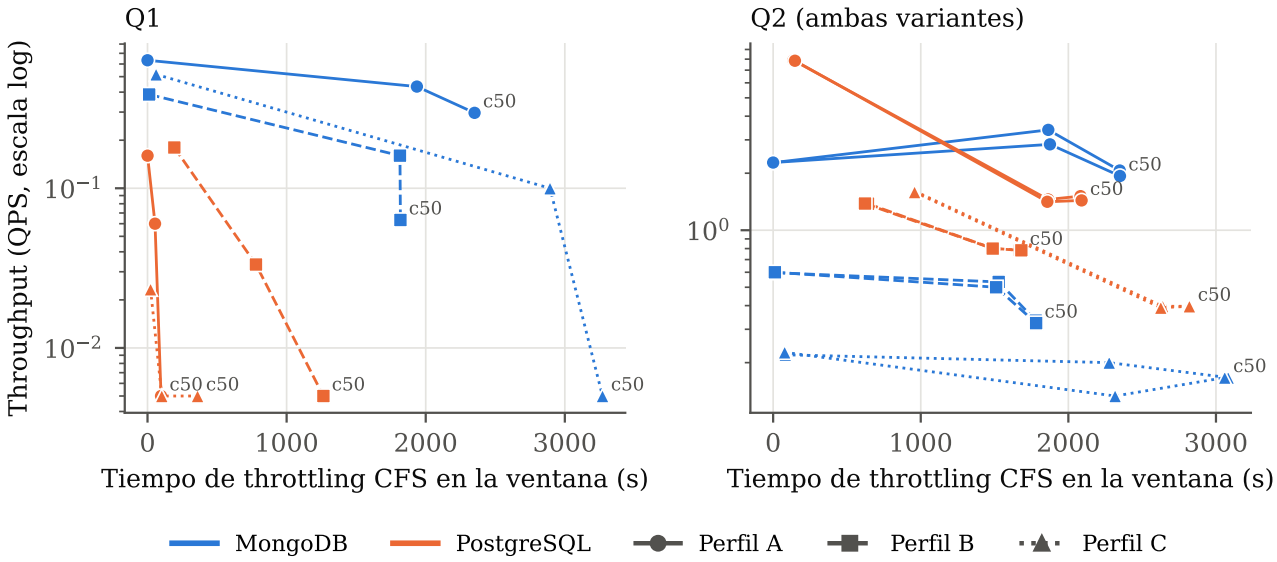


Figura 2: Throttling CFS acumulado en la ventana frente a throughput (mediana por celda). Cada trazo recorre las concurrencias 1, 10 y 50 de un motor y un perfil; el final del trazo está etiquetado. Las celdas con 0 QPS se dibujan en el borde inferior.

En la figura 2, más concurrencia aumenta el throttling sin aumentar el throughput: la cuota ya está agotada, y el trabajo extra solo se convierte en cola. En Q1 en A y C, el throttling de PostgreSQL es bajo (menos de 400 s) incluso cuando no termina ninguna consulta: el cuello de botella no es la CPU sino la escritura y lectura de los archivos temporales del ordenamiento. En B, PostgreSQL Q1 sí se acerca a su cuota (558 de 600 CPU-s con 50 clientes) y acumula 780–1 264 s de throttling. B corrió solo en su equipo, mientras que A y C compartían el disco donde volcaban sus temporales; es consistente con que el disco compartido frenara a PostgreSQL en la ronda 1, aunque con un solo equipo para B no se puede separar del efecto del hardware.

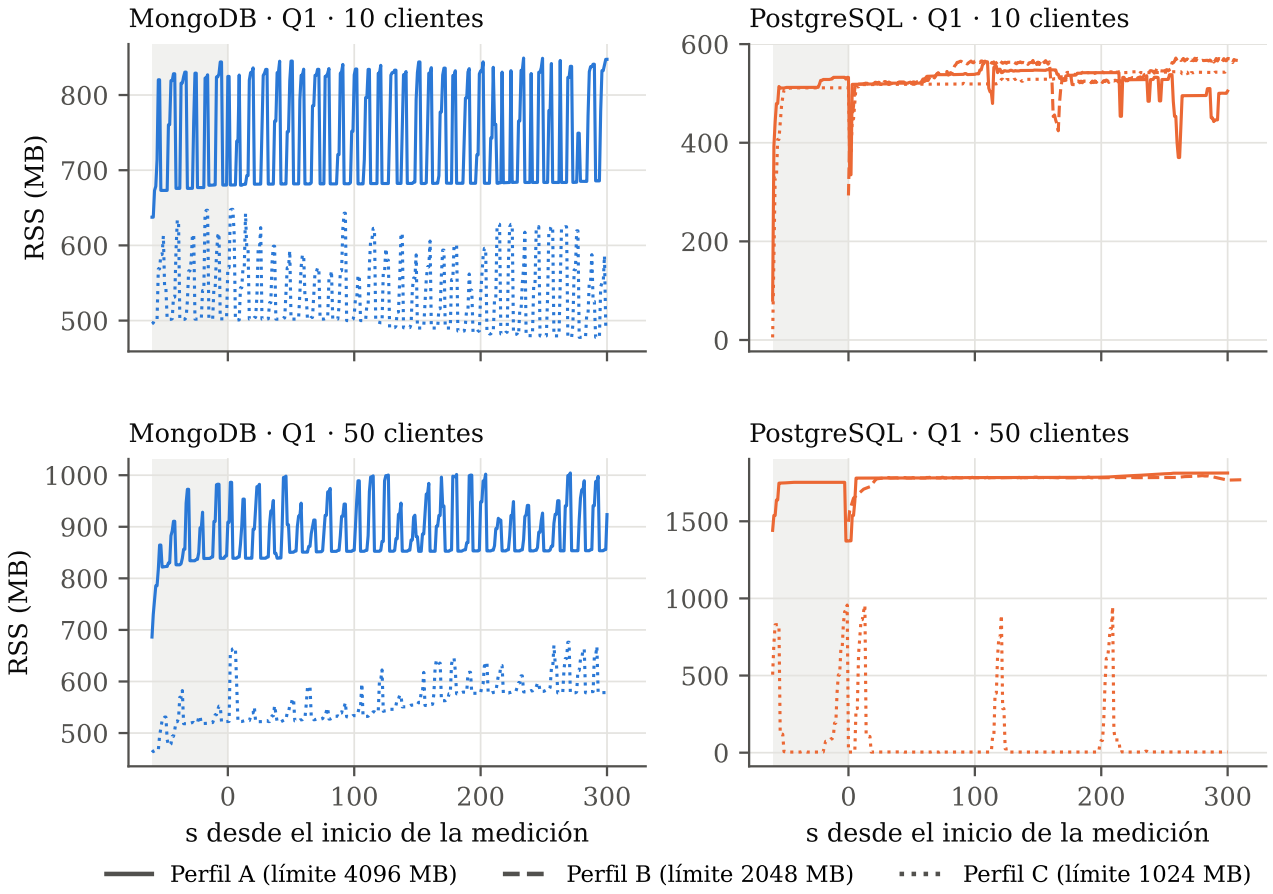


Figura 3: Curva de RSS del contenedor durante Q1 (repetición 1), cada 1 s desde Prometheus o desde la cgroup. La banda gris es el warmup. MongoDB en B no tiene curva por la falta de serie descrita en la sección 7.

La figura 3 muestra el patrón de memoria de cada motor. MongoDB oscila en dientes de sierra, del orden de 150 MB por consulta, por encima de su caché WiredTiger, y nunca se acerca al límite. En PostgreSQL con 1 GB y 50 clientes el RSS sube hasta ∼950 MB y cae a cero en cada `oom_kill`: el *postmaster* reinicia los procesos y entra en *recovery* varias veces dentro de la misma ventana. Con 4 GB, el RSS de PostgreSQL se estabiliza en ∼1.8 GB; con 2 GB (B) llega a la misma cifra, a 250 MB del límite, sin llegar al OOM.

5. Planes de ejecución

Los 18 planes (2 motores × 3 consultas × 3 perfiles) están en `results/explain/`: `explain("executionStats")` en MongoDB y `EXPLAIN (ANALYZE, BUFFERS)` en PostgreSQL, capturados con un solo cliente.

Tabla 8: Resumen de los planes en el perfil A (los perfiles B y C tienen la misma forma).

Consulta	Motor	Plan y memoria
Q1	MongoDB	`COLLSCAN` de 500 000 documentos → `$unwind` (1 500 160) → `$match` (974 583) → `$group` (180 grupos). Memoria de agrupación: 128 KB. `usedDisk: false`, 0 spills. 2.2 s en A, 2.3 s en C.
Q1	PostgreSQL	`Parallel Seq Scan` (2 workers) → `Sort` de filas completas (ancho estimado 1 304 B) con **external merge**: 192 + 235 + 226 MB en disco → `Gather Merge` → `jsonb_array_elements` → `Incremental Sort` (external merge) → `GroupAggregate`. **Temp: 187 865 bloques escritos** (≈1.5 GB). El planificador estima 16.5 millones de grupos frente a 180 reales. 10.0 s en A, 20.6 s en C.
Q2	MongoDB	`IXSCAN` sobre `idx_cerrado_en` (250 141 claves) → `FETCH` y filtro (212 641) → `SORT` top-k con límite 1000 (`memLimit` 100 MB, pico 1.09 MB) → `PROJECTION`. Sin disco con `allowDiskUse` false ni true. 0.45 s en A, 5.5 s en C.
Q2	PostgreSQL	`Parallel Seq Scan` (2 workers, 500 000 filas; no usa el índice de fecha porque el filtro conserva ∼42 % de la tabla) → `Sort` **top-N heapsort** de 272 KB por proceso → `Gather Merge` → `Limit`. Igual con `work_mem` de 4 MB y de 64 MB. 0.12 s en A, 0.88 s en C.

Los planes respaldan los cuellos de botella medidos:

- **Q1 en PostgreSQL**: con el `work_mem` por defecto, un ordenamiento de ∼650 MB obliga a usar disco. El volumen temporal de una consulta en el plan (≈1.5 GB) es coherente con lo medido: 42.5 GB en una ventana de A con un cliente, que terminó 48 consultas (≈0.9 GB por consulta, más las que quedaron a medias). La mala estimación de grupos lleva al planificador a ordenar en lugar de usar un `HashAggregate` de 180 grupos.
- **Q1 en MongoDB**: el pipeline procesa en flujo y su agrupación ocupa kilobytes, así que solo lo limita la CPU, como confirma el throttling (sección 6).
- **Q2**: el Top-1000 nunca necesita más de ∼1 MB en ningún motor, así que las variantes `allowDiskUse` y `work_mem` no cambian el plan ni el resultado. La especificación pide reportar las variantes con y sin spill; en Q2 ambas resultan sin spill, y el contraste con volcado a disco lo aporta Q1.

6. Interpretación: memoria, volcado a disco y throttling

M1. ¿Cómo cambian P95 y P99 al limitar CPU y memoria?

- **Con un cliente** el efecto es moderado cuando la consulta usa un solo núcleo (MongoDB Q1: P99 de 1.6 s en A a 2.3 s en C, ×1.4) y fuerte cuando la consulta usa varios núcleos o disco. MongoDB Q2 sube de 0.58 a 5.4 s (×9.3) y PostgreSQL Q1 de 11.3 a 51.9 s (×4.6), porque en C sus workers paralelos y su I/O temporal compiten por un solo núcleo.
- **Con concurrencia** el P99 lo dominan la cola y el throttling. Con 10 clientes la cuota se agota en los tres perfiles (A: 1 200 CPU-s de 1 200 posibles; B: ∼580 de 600; C: 300 de 300) y el P99 crece en proporción al número de clientes por núcleo.
- **P95 y P99 quedan casi pegados** cuando el sistema está saturado, como en MongoDB Q1 con 10 clientes en C (P95 113.2 s, P99 113.3 s): todas las consultas esperan lo mismo y la cola de la distribución desaparece. La degradación se ve también en el P50.

- **El límite de memoria cambia el resultado**, y no solo la latencia, en PostgreSQL Q1: con 1 GB y 50 clientes el contenedor muere (28 `oom_kill` en 3 repeticiones).
- **En B** el efecto con concurrencia es intermedio entre A y C: el P99 de MongoDB Q1 con 10 clientes es 73.6 s (34.9 en A, 113.3 en C) y el de PostgreSQL Q2 con 10 clientes, 21.7 s (13.9 en A, 44.1 en C). Con un cliente B no es intermedio porque su equipo tiene núcleos más lentos; la comparación que aísla el efecto del límite es A frente a C, medidos en el mismo hardware.

M2. ¿Qué motor gestiona mejor los buffers antes de spill u OOM?

- **En la agregación con desanidado (Q1), MongoDB.** Agrupa en flujo con un estado de ∼128 KB, nunca vuelca a disco y no tuvo ningún OOM en ningún perfil. Con 1 GB y 50 clientes falla por tiempo (CPU), no por memoria.
- **PostgreSQL vuelca a disco en todas las corridas de Q1** que llegan a ejecutar consultas: el ordenamiento de filas completas no cabe en 4 MB de `work_mem`. Escribe 6.7–7.1 GB por ventana en C, 13.5–47.6 GB en B y 22.9–42.5 GB en A, y con 50 backends en 1 GB termina en OOM: cada backend y sus workers reservan memoria propia para ordenar y descomprimir (*detoast*) el JSONB, y la suma rebasa el límite.
- **En el Top-K (Q2) los dos gestionan bien el buffer.** Ambos usan un ordenamiento top-k acotado (1.09 MB en MongoDB, 272 KB por proceso en PostgreSQL), muy por debajo de `memLimit` (100 MB) y de `work_mem` (4 MB). Ninguna de las 108 corridas de Q2 (54 por motor, en los tres perfiles) presentó spill u OOM.

M3. Huella de CPU y memoria por cada 1 000 documentos

Se definen como:

$$\text{CPU-s}/1000 = \frac{\text{CPU-s de la ventana}}{\text{consultas terminadas} \times 500\,000} \times 1000, \qquad \text{MB}/1000 = \frac{\text{RSS máximo}}{500\,000} \times 1000.$$

El denominador son los 500 000 documentos de la colección que evalúa cada consulta. Se usa la concurrencia 1, donde el costo por consulta no incluye tiempo de cola. Los CPU-s salen de `container_cpu_usage_seconds_total` (Prometheus) o de `usage_usec` de la cgroup (`results/series/`).

Tabla 9: M3 con un cliente: CPU-s y MB de RSS por cada 1 000 documentos (mediana de 3 repeticiones). † Estimado: sin serie de CPU para esa celda, se usa la cuota del perfil o el consumo de la misma celda en A, el menor de los dos.

		Perfil A		Perfil B		Perfil C	
Motor	Query	CPU-s	MB	CPU-s	MB	CPU-s	MB
MongoDB	Q1	0.0034	1.587	0.0056†	1.672	0.0039	1.258
MongoDB	Q2_DISK0	0.0010	2.022	0.0039	1.881	0.0091	1.478
MongoDB	Q2_DISK1	0.0009	2.000	0.0039	1.871	0.0088	1.440
PostgreSQL	Q1	0.0091	0.204	0.0139	0.217	0.0128	0.203
PostgreSQL	Q2_WM4	0.0010	0.028	0.0028	0.040	0.0012	0.032
PostgreSQL	Q2_WM64	0.0010	0.029	0.0028	0.039	0.0013	0.032

- **CPU**: en Q1, MongoDB gasta ∼0.0034 CPU-s por 1 000 documentos y PostgreSQL ∼0.0091 en A, 2.7 veces más, a pesar de usar menos CPU en total: PostgreSQL pasa la mayor parte del tiempo

esperando disco y termina menos consultas. En Q2 los dos están en ∼0.001 en A; en C, MongoDB sube a 0.009 (el `FETCH` de 250 141 documentos con un núcleo) y PostgreSQL se mantiene en 0.0012. En B, MongoDB Q2 queda entre ambos (0.0039) y PostgreSQL Q2 por encima de los dos (0.0028): con 2 CPU sus 3 procesos paralelos agotan la cuota con un solo cliente sin terminar más consultas. El valor de MongoDB Q1 en B (0.0056) es una cota superior estimada.

- **Memoria**: el RSS de MongoDB (1.3–2.0 MB por 1 000 documentos) incluye su caché WiredTiger, que vive en memoria anónima. El de PostgreSQL (0.03–0.2 MB) no incluye `shared_buffers`, que es memoria compartida. Las cifras de memoria no son un costo marginal por consulta, sino la huella del proceso con el límite dado.

Throttling de cgroups

Con 10 o más clientes ambos motores consumen casi exactamente su cuota de CPU (1 200 CPU-s en A, 600 en B y 300 en C por ventana), y el throttling acumulado va de 1 850 a 3 280 s en A y C y de 1 480 a 1 820 s en B. Es más que la ventana porque se suma por cada CPU en la que había hilos esperando. La excepción es PostgreSQL Q1, con menos de 400 s de throttling en A y C: no espera CPU sino disco temporal. En B llega a 780–1 264 s (sección 4). PostgreSQL Q2 sufre throttling incluso con un cliente (137 s en A, 642 en B, 958 en C), porque su consulta paralela usa 3 procesos y con 2 o 1 CPU satura la cuota sola.

Respuestas a las preguntas del revisor

1. **¿El baseline es comparable?** Sí, en dataset y consultas: el mismo NDJSON (sha256 verificado), la misma validación y resultados equivalentes en ambos motores (sección 2). En hardware, A y C comparten equipo y B no (sección 7).
2. **¿P50/P95/P99 o promedios?** Percentiles sobre las latencias individuales de cada corrida (tablas 4–6); las latencias crudas están en `results/raw/`.
3. **¿Hubo throttling CFS? ¿Cuántos segundos?** Sí, en todas las celdas con 10 y 50 clientes, y con 1 cliente en C y en PostgreSQL Q2. Hasta 3 280 s por ventana (columna “Throttled” y figura 2).
4. **¿Spill u OOM? ¿En qué perfil y concurrencia?** Spill en PostgreSQL Q1 en todos los perfiles y concurrencias; OOM en PostgreSQL Q1, perfil C, 50 clientes. MongoDB y Q2 sin spill (tabla 7, evidencia en `results/evidencia_spill_oom/`).
5. **¿Cuántas repeticiones y cómo se resumen?** 3 por celda, mediana con rango mínimo–máximo.
6. **¿El plan respalda el cuello de botella?** Sí: el external merge de PostgreSQL Q1 explica su spill y su bajo uso de CPU; el pipeline en flujo de MongoDB explica que solo lo limite la CPU; el top-k de ambos explica la ausencia de spill en Q2 (sección 5).

7. Recomendaciones técnicas para despliegues documentales en la nube

1. **Dimensionar la CPU por el P99, no por el promedio.** Con cuotas CFS estrictas, el P99 crece en proporción a los clientes por núcleo una vez agotada la cuota. Conviene vigilar `throttled_usec` (o `container_cpu_cfs_throttled_seconds_total`) como señal de saturación junto al P99. En cargas sensibles a latencia, valorar no imponer límite de CPU o reservar núcleos (*requests* cercanos al límite).

2. **Limitar la concurrencia en PostgreSQL con un *pooler*** (PgBouncer o el de la aplicación). La memoria de PostgreSQL crece con el número de backends y sus workers; con contenedores pequeños, el OOM llega por concurrencia antes que por tamaño de datos. Ajustar `max_parallel_workers_per_gather` al número de CPU del contenedor.
3. **Revisar `work_mem` y el plan de las agregaciones JSONB con desanidado.** La estimación por defecto de `jsonb_array_elements` lleva a ordenar filas completas a disco. Opciones: subir `work_mem` por sesión solo para esas consultas (con el límite del contenedor en mente: work_mem× procesos), reescribir la consulta para proyectar las claves antes de agrupar, o normalizar el arreglo en una tabla propia si la agregación es frecuente.
4. **MongoDB para agregaciones en flujo con memoria acotada.** Su pipeline mantuvo un estado de kilobytes y nunca hizo spill; su límite es la CPU: escalar núcleos mejora el throughput con concurrencia, no la latencia de una consulta aislada (un hilo por consulta).
5. **Sin swap y con margen de memoria.** Con `memory.swap.max = 0` la presión de memoria se manifiesta como OOM visible y no como latencia oculta; dejar margen para la caché de WiredTiger o los `shared_buffers` más la memoria por conexión.
6. **Índices y Top-K.** Un Top-K con `LIMIT` se resuelve con un ordenamiento acotado en ambos motores; no es necesario indexar el campo de orden para evitar el spill. Sí conviene un índice selectivo para el filtro: MongoDB lo usa (250 141 claves); a PostgreSQL no le compensa con una selectividad del 42 %.

Amenazas a la validez

- **Equipos distintos.** B se midió en otro equipo (i5-1135G7 de 11.ª generación frente al i5-13420H de 13.ª), así que su comparación con A y C mezcla el efecto del límite con el del hardware. Se nota con un cliente, donde B resulta a veces más lento que C. La comparación entre motores dentro de cada perfil sí es homogénea.
- **Perfiles en paralelo.** A y C compartieron equipo; sus límites no se solapan (5 de 12 hilos, 5 de 7.6 GB), pero comparten caché L3, memoria y disco. El disco compartido afecta sobre todo a PostgreSQL Q1, que escribe decenas de GB temporales por ventana; B, que corrió solo, muestra más uso de CPU en esa celda.
- **Telemetría en el equipo de B.** En ese equipo, cAdvisor no exponía las métricas por contenedor, así que el programa de medición pasó a leer la CPU y el RSS directamente de la cgroup de cada contenedor, con el mismo significado. Las 9 corridas de MongoDB Q1, las primeras de B, se hicieron antes de ese cambio y no tienen serie de CPU ni de RSS: su M3 de CPU es una cota († en la tabla 9), mientras que su throttling y su RSS máximo sí se midieron.
- **Ventana censurada.** Cuando una consulta dura más que la ventana, la latencia no se registra y el P99 subestima la cola; esas celdas se marcan `error` con 0 QPS. Los CPU-s de la ventana incluyen el trabajo de consultas cortadas, lo que encarece M3 en las celdas saturadas (por eso M3 usa 1 cliente).
- **Telemetría propia.** El *healthcheck* de MongoDB (`mongosh` cada 5 s) corre dentro de la cgroup y consume algo de CPU en todos los perfiles.
- **Virtualización.** WSL2 añade una capa de virtualización; el flush de caché vacía la page cache del kernel de la VM, no la del host Windows.
- **Tres repeticiones** dan un rango, no un intervalo de confianza.

A. Reproducción y artefactos

- Repositorio: `https://github.com/lsprz2773/BDN-Exp`. Preparación: `scripts/setup_env.sh <perfil>`; ronda: `scripts/run_benchmarks.sh -profile <P> -round <N>` (con `-parties` y `-session` para medir dos perfiles en paralelo). Las instrucciones completas están en el `README.md`.
- Mediciones: `results/csv/mediciones_ronda1.csv` y `mediciones_ronda2.csv`, con las 14 columnas de la especificación y el mismo encabezado.
- Latencias crudas, series de RSS/CPU, planes, entorno y evidencia de spill/OOM en `results/raw`, `results/series`, `results/explain`, `results/entorno` y `results/evidencia_spill_oom`.
- Tablas y figuras de este reporte: `scripts/analyze_results.py`.

Ejemplo de fila del CSV (perfil C, PostgreSQL Q1, 1 cliente):

```
timestamp_epoch,engine,profile,query_id,concurrency,run_number,throughput_qps,latency_p50_ms,
    latency_p95_ms,latency_p99_ms,cpu_throttled_seconds,max_rss_mb,disk_spill,status
1793571192,postgresql,PROFILE_C_SEVERE,Q1,1,1,0.0333,27021.55,38973.8,40516.35,25.767,101.4,true,spill
```